



# *AGRI API Style Guide*

## *Informatieronde*

---

AgroConnect

Bernard van Raaij | Van Raaij Advies | 25 juni 2026

# AgroConnect standaardisatie in de API wereld

01

**Toegangsbeleid standaarden**

*Wie bepaalt, wie gebruikt*



02

**Life Cycle Management**

*Beheer door de tijd*



03

**Open Source / GitHub**

*Decentrale samenwerking*



04

**AGRI API Style Guide**

*Uniformiteit in API-bouw*



# 04 AGRI API Style Guide

*Wildgroei · Inefficiency · Kennishiaat*

## Wat is het?

Een set richtlijnen voor de technische inrichting van een API-platform, onafhankelijk van de specifieke functionaliteit.

Toepasbaar voor elk API-platform in de Agri- en Foodketen.

## Voordelen



Meer uniformiteit in API-platforms



Herkenbaarheid voor client-bouwers



Hogere begripelijkheid van uitgewisselde informatie



Minder kans op mis-interpretaties

# Wat bereik je met een Style Guide?



**Consistentie & Leesbaarheid**



**Snellere onboarding**



**Minder fouten & kosten**



**Herbruikbaarheid**



**Betere interoperabiliteit**



**Hogere datakwaliteit**

*Doel: client-partijen kunnen een koppeling bouwen puur op basis van de gepubliceerde API-documentatie.*

# Beleid: aansluiting bij NL API Strategie & ADR

## Kennisplatform APIs

*Kadaster, VNG, KvK,  
Geonovum, Logius*

## NL API Strategie + ADR

*API Design Rules  
Normatieve Modules*

## Forum Standaardisatie

*'Pas toe of leg uit-lijst'*

## AGRI API Style Guide

*AgroConnect B2B*

### Keuze AgroConnect

Gebruik wat bestaat en vind het wiel niet opnieuw uit!

AgroConnect sluit aan bij de ADR en maakt een eigen variant die:

- De ADR-regels grotendeels overneemt
- Een beperkt aantal regels aanpast waar nodig (mn voorbeelden)
- Extra regels toevoegt die wij nuttig vinden maar niet (nog) in de ADR zitten



# Roadmap: van v0.1 naar v1.0

*Open source aanpak — draagvlak is essentieel*

1

## Informatierondes

Uitnodiging aan bedrijven & solution providers. Toelichting achtergrond en individuele regels.



2

## Consultatieperiode

Iedereen kan commentaar leveren, issues aankaarten of verbeteringen voorstellen via GitHub Issues.



3

## Publicatie v1.0

Na afhandeling van alle issues: vaststelling en publicatie op GitHub. Commitment van deelnemers verwacht.



# Over deze informatieronde

## *Achtergrond en aanleiding*

### **Waarom deze sessies?**

U heeft zich aangemeld om mee te denken over de doorontwikkeling van de AGRI API Style Guide (AASG).

Met deze richtlijn werken we aan meer uniformiteit in API-platforms in de agri- en foodketen.

Draagvlak en praktijkkennis zijn essentieel. Daarom lichten we de AASG toe en leggen we uit hoe we samenwerken via GitHub.

.

### **Wat verwachten we van u?**

#### **Voorkennis**

Enige voorkennis van OpenAPI, REST en GitHub helpt om goed aan te sluiten

#### **GitHub-account**

Gratis aan te maken op [github.com](https://github.com). Nodig om Issues in te dienen en mee te werken aan de documentatie.

#### **Issues indienen**

Via GitHub Issues kunt u commentaar leveren, vragen stellen en verbeteringen voorstellen.

#### **Commitment bij v1.0**

Na vaststelling verwachten we dat deelnemers de AASG ook daadwerkelijk toepassen in hun API-platforms.

# AgroConnect op GitHub



*Wat we al doen en wat er aankomt — [github.com/AgroConnectNL](https://github.com/AgroConnectNL)*

## Beschikbare repositories

### API-Style-Guide (publiek)

AGRI API Style Guide documentatie. Licentie: CC-BY-4.0.  
Zojust publiek gemaakt!  
[github.com/AgroConnectNL/API-Style-Guide](https://github.com/AgroConnectNL/API-Style-Guide)

### eDairy (productgroep Rund)

Beheerd door AgroConnect-Dairy. Aparte organisatie op GitHub.  
[github.com/AgroConnect-Dairy/Home](https://github.com/AgroConnect-Dairy/Home)

### Licentie

Alle publieke AgroConnect-documentatie valt onder de Creative Commons CC-BY-4.0 licentie. Gebruik is vrij mits bronvermelding.

## In ontwikkeling/overweging

### Codelijsten (AgroConnect)

Centrale repository voor AgroConnect-codelijsten. Nog in opbouw.

### eCrop (productgroep Teelt)

Documentatie van de AgroConnect eCrop standaard.  
Toegankelijk voor leden van de productgroep.  
[github.com/AgroConnectNL/eCrop](https://github.com/AgroConnectNL/eCrop)

### ePoultry, ePigs en andere productgroepen

Repositories voor de productgroepen Pluimvee, Varken en Transactie zijn nog in constructie.

### Toegang tot productgroep-repositories

Toegang is beperkt tot leden van de betreffende productgroep. Aanmelden via [info@agroconnect.nl](mailto:info@agroconnect.nl)



# GitHub als samenwerkingsomgeving voor de AASG



## *Rollen, rechten en werkwijze*

### Wie doet wat?

#### **AgroConnect (beheerder)**

Eigenaar van de repository. Beoordeelt en verwerkt alle voorstellen. Behoudt het recht om Issues te sluiten of af te wijzen.

#### **Deelnemers (contributors)**

Iedereen met een GitHub-account kan Issues aanmaken, reageren op Issues van anderen, en discussie voeren in de commentaren.

#### **Werkgroepleden**

Kunnen worden uitgenodigd als collaborator, met schrijfrechten op specifieke branches of voor het reviewen van pull requests.

#### **Toegang**

De AASG-repository is publiek leesbaar. Bijdragen vereisen een gratis GitHub-account.

### Spelregels

#### **Taal**

Issues en discussies bij voorkeur in het Nederlands. Technische termen (regelcodes, HTTP-terminologie) in het Engels conform de AASG.

#### **Eén Issue per onderwerp**

Maak een apart Issue aan voor elk afzonderlijk commentaar, vraag of verbetervoorstel. Combineer geen meerdere punten in één Issue.

#### **Constructief en concreet**

Beschrijf wat het probleem is, waarom het een probleem is, en — indien mogelijk — hoe het opgelost zou kunnen worden.

#### **Geen directe wijzigingen in main**

Wijzigingen via pull requests. De main branch is beschermd en kan alleen worden gewijzigd door AgroConnect.

# GitHub Issues: hoe dien je een bijdrage in?



*Stap voor stap — [github.com/AgroConnectNL/API-Style-Guide/issues](https://github.com/AgroConnectNL/API-Style-Guide/issues)*

## Een Issue aanmaken

### Stap 1 — Ga naar de repository

Navigeer naar [github.com/AgroConnectNL/API-Style-Guide](https://github.com/AgroConnectNL/API-Style-Guide) en klik op het tabblad Issues.

### Stap 2 — Klik op New Issue

Kies een passend label (bijv. vraag, verbetervoorstel, bug) als dat beschikbaar is.

### Stap 3 — Geef een duidelijke titel

Verwijs bij voorkeur naar de regelcode, bijv.: [M006] Vraag over major-versie in header vs. URI

### Stap 4 — Beschrijf het Issue

Wat is het probleem of de vraag? Welke regel betreft het?  
Wat is jouw suggestie of verwachting?

### Stap 5 — Verzend het Issue

Klik op Submit new issue. AgroConnect bevestigt ontvangst en reageert via het Issue zelf.

## Wat gebeurt er daarna?

### Beoordeling door AgroConnect

AgroConnect leest elk Issue en kent een label toe: geaccepteerd, in behandeling, wachten op feedback, of afgewezen.

### Discussie in de commentaren

Andere deelnemers kunnen reageren. De discussie blijft openbaar en transparant zichtbaar voor iedereen.

### Verwerking in de documentatie

Geaccepteerde Issues worden verwerkt in de Markdown-documentatie. Het Issue wordt gesloten met een verwijzing naar de betreffende wijziging.

### Mijlpalen

AgroConnect koppelt Issues aan mijlpalen (bijv. v0.9, v1.0) zodat duidelijk is wanneer een punt wordt opgepakt.

# AASG: 48 regels in 6 categorieën

*Opbouw van de AGRI API Style Guide*

## De zes categorieën

### **M — Meta-informatie & Versiebeheer**

M001–M010 · 10 regels

### **S — Security**

S001–S006 · 6 regels

### **U — URLs & Resources**

U001–U007 · 7 regels

### **R — RESTful principles**

R001–R005 · 5 regels

## Vervolg

### **P — Payloads**

P001–P013 · 13 regels

### **H — HTTP-methoden & Responses**

H001–H008 · 8 regels

### **Relatie met NLGov ADR**

Veel regels zijn overgenomen of aangepast vanuit de NLGov REST API Design Rules. Enkele regels zijn specifiek voor AgroConnect.

# M — Meta-informatie & Versiebeheer [M001–M010]

*De API-specificatie als contract met de buitenwereld*

## Kernregels

### M001 — OpenAPI verplicht

Elke API-specificatie MOET worden beschreven en gepubliceerd in OpenAPI (bij voorkeur als YAML).

### M002 — Meta-informatie

Titel, versie, beschrijving en contactgegevens zijn verplicht in de specificatie.

### M003 — Taal

De specificatie MOET in Amerikaans Engels worden geschreven.

### M004 — Semantic versioning

Versiebeheer via major.minor.patch. Major = breaking change; minor = nieuwe features; patch = bugfix.

## Response-headers & beheer

### M005 — Max. 2 major versies

Maximaal twee major-versies tegelijk in productie, met een vaste transitieperiode en changelog.

### M006 — Versie in request-header

Major-versie via de header Major-Version: 1. Niet in de URL (afwijking van de ADR).

### M007 — Client software in User-Agent

Identificatie van client software in User-Agent

### M008 — Volledige versie in response

De server geeft altijd de volledige versie terug in de API-Version response-header.

### M009 & M010

Unieke Request-Id en Request-Date-Time verplicht in elke response, voor tracing en debugging.

# S — Security [S001–S006]

## *Beveiliging op transport- en HTTP-niveau*

### Transport & URI

#### **S001 — TLS verplicht**

Alle verbindingen MOETEN via TLS lopen, conform de NCSC TLS-richtlijnen 2025. Geen uitzonderingen.

#### **S002 — Geen gevoelige data in URI**

URI's worden gecached en gelogd. Ze MOGEN GEEN wachtwoorden, BSN's of andere gevoelige informatie bevatten.

### HTTP-niveau security

#### **S003 — Security-headers verplicht**

Elke response MOET security-headers bevatten, zoals:  
Cache-Control: no-store  
Strict-Transport-Security  
X-Content-Type-Options: nosniff  
X-Frame-Options: DENY

#### **S004 — CORS**

Cross-origin toegang MOET worden beperkt via een allowlist in de Access-Control-Allow-Origin header. Gebruik van wildcard (\*) wordt AFGERADEN.

### Browser clients en Content validatie

#### **S005 — Browser-based apps**

Browser-apps MOETEN de OAuth 2.0 best practices voor browser-based apps volgen (IETF draft).

#### **S006 — Content-type validatie**

Request/response body MOET overeenkomen met de Content-Type header. Ongeldige types retourneren HTTP 415.

# U — URLs & Resources [U001–U007]

## *Naamgeving en structuur van API-adressen*

### Naamgeving

#### **U001 — Geen /api of /services**

Gebruik nooit /api of /services als base path. Resources starten direct onder het rootpad /

#### **U002 — Zelfstandige naamwoorden**

Resourcenamen moeten zelfstandige naamwoorden zijn, geen werkwoorden (niet /getgrowers maar /growers).

#### **U003 — Meervoud**

Collections moeten altijd in het meervoud: /growers, /crops, /tasks.

#### **U004 — kebab-case in padsegmenten**

Padsegmenten: alleen kleine letters, cijfers en koppeltekens. Bijv. /inbound-deliveries

### Parameters & hiërarchie

#### **U005 — Geen trailing slashes**

Paden mogen geen dubbele of afsluitende slashes bevatten. /growers/ geeft HTTP 404.

#### **U006 — lowerCamelCase query parameters**

Query-parameters in lowerCamelCase:  
?deliveryDate=2026-03-25 (niet ?delivery-date=...)

#### **U007 — Hiërarchische relaties in URI**

Parent-child relaties MOETEN in het pad staan:  
/growers/{id}/crops/{id}/tasks  
Identifiers van parent-resources mogen niet in de payload staan als ze al in de URI zitten.

# R — RESTful principes [R001–R005]

## *Architectuurprincipes voor interoperabele API's*

### **RESTful principes**

#### **R001 — Stateless**

API's MOETEN stateless zijn. Elke request bevat alle benodigde informatie. De server bewaart geen sessie-informatie.

#### **R002 — Volledige representatie in response**

Elke response moet de volledige resource-representatie bevatten, inclusief alle velden. Geen partieel antwoord bij succesvolle verwerking.

#### **R003 — Server-side UUID als identifier**

Bij aanmaken (POST) moet de server een uniek id (UUID) toekennen en teruggeven in de response. Vervolgrequest gebruiken dit id in de URI.

#### **R004 — Verberg implementatiedetails**

De API mag geen implementatiedetails blootgeven (frameworks, databasemodellen, interne naamgeving). De API-structuur MOET client-vriendelijk zijn, niet een 1-op-1 mapping van het datamodel.

#### **R005 — Client-side caching (optioneel)**

GET-operaties mogen caching ondersteunen via Cache-Control, ETag en Last-Modified headers. Andere methoden (POST, PUT, PATCH, DELETE) mogen niet gecached worden.

#### **Bron**

Gebaseerd op de zes REST-principes van Roy Fielding, grotendeels overgenomen uit de NLGov ADR.

# P — Payloads [P001–P013]

## *Structuur en opmaak van request- en response-inhoud*

### **Data-formaat & naamgeving**

#### **P001 & P002 — JSON verplicht**

Payloads moeten JSON zijn (RFC 7159). Media types: application/json, application/json-patch+json of application/problem+json.

#### **P003 — Schemanamen enkelvoud**

Schemanamen moeten enkelvoud zijn (InboundDelivery, niet InboundDeliveries).

#### **P004 — PascalCase schemanamen**

Alle schemanamen in PascalCase (CamelCase): InboundDeliveryDetail

#### **P005 & P006 — lowerCamelCase properties**

Properties in lowerCamelCase. Array-properties in meervoud (thirdPartyIds), objecten enkelvoud (product).

### **Datum, PATCH & fouten**

#### **P007 Null en absente properties**

Worden hetzelfde behandeld

#### **P008 - P010 — Datum/tijd**

Datums zonder tijd: format: date. Datum+tijd: format: date-time, RFC 9557/ISO 8601. UTC (Z) verplicht in responses. Alle offsets geaccepteerd in requests.

#### **P011 — GET en DELETE hebben geen payload**

GET en DELETE mogen geen request payload hebben

#### **P012 — PATCH via JSON Patch**

PATCH moet RFC 6902 JSON Patch gebruiken als payload

#### **P013 — Problem Details bij fouten**

Foutresponses (4xx/5xx) MOETEN RFC 9457 Problem Details bevatten



# H — HTTP-methoden & Responses [H001–H008]

*Correct gebruik van HTTP-semantiek*

## Methoden & veiligheid

### H001 — Alleen standaard HTTP-methoden

Uitsluitend GET, POST, PUT, PATCH en DELETE. PATCH is toegestaan. POST alleen op collections. PUT/PATCH/DELETE alleen op singletons.

### H002 — Safety & idempotency

GET, HEAD en OPTIONS: veilig én idempotent. PUT en DELETE: idempotent maar niet veilig. POST en PATCH: geen van beide.

### H003 & H004 — Statuscodes

Verplichte statuscodes per operatie:

GET collection: 200, 400, 401, 500

GET singleton: 200, 400, 401, 404, 500

POST: 201/202, 400, 401, 403, 500

PUT/PATCH: 200/202, 400, 401, 403, 404, 500

DELETE: 204/202, 400, 401, 403, 404, 500

## Statuscodes & foutafhandeling

### H005 — PUT is geen insert or replace

PUT u niet bestaande resource geeft 404 en geen alternatieve insert/create

### H006–H008 — Foutcodes

400 Bad Request: ongeldige input of schema-fouten (alle fouten tegelijk retourneren, H007).

401 Unauthorized: uitsluitend voor authenticatiefouten.

403 voor autorisatiefouten.

# Dank voor uw aandacht!

*Vragen?*

---

De content van deze presentatie is geheel bedacht door Bernard van Raaij.  
De opmaak is verzorgd door Claude van Anthropic.

